

DATABASES

# Laboratory work 4

pgAdmin query screenshots

Student

**Madi Murat**

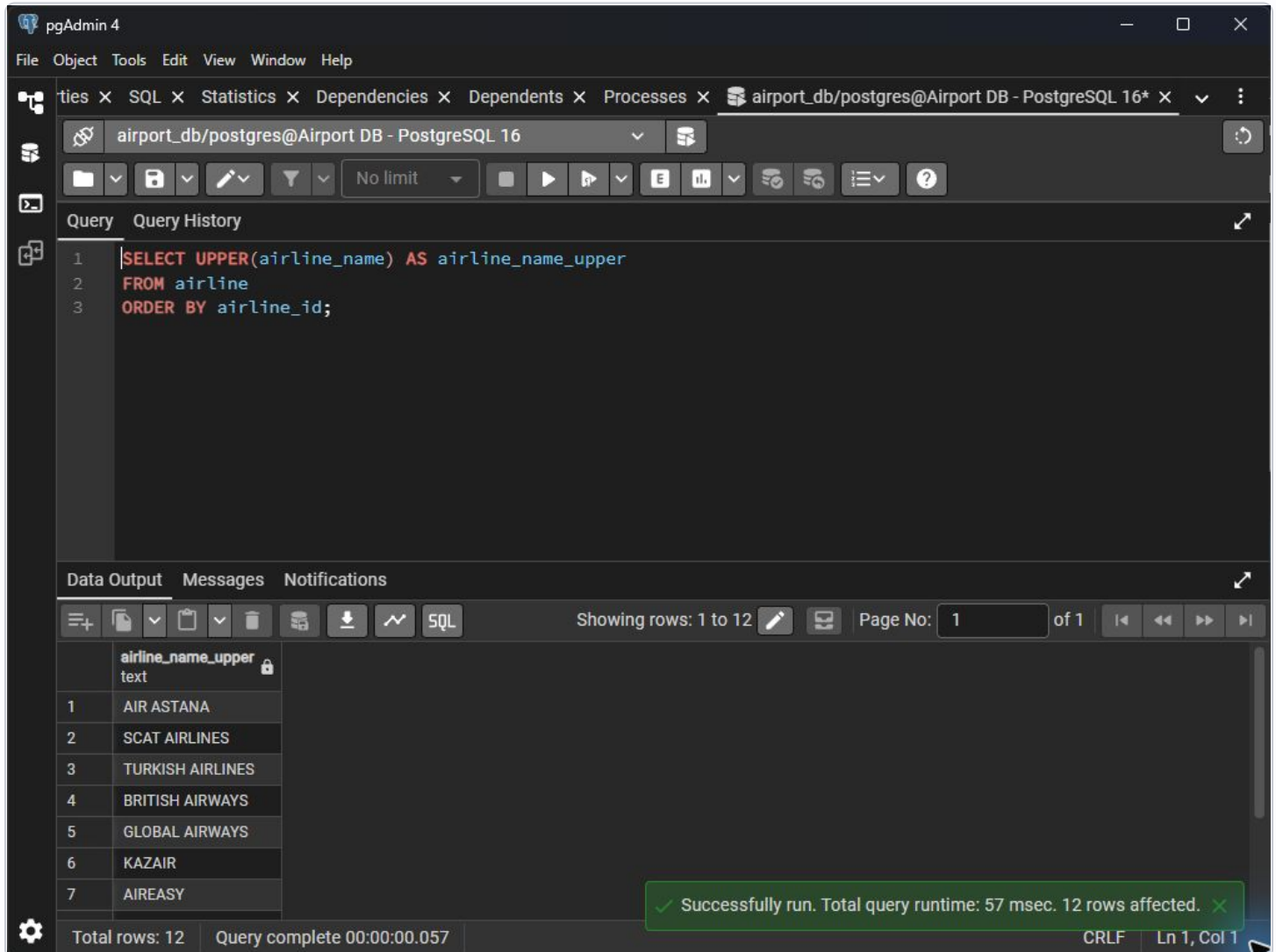
Database airport\_db

PostgreSQL 16

04

# Task 1

Retrieve all airline names in uppercase.



The screenshot shows the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window displays the 'airport\_db/postgres@Airport DB - PostgreSQL 16\*' connection. The 'Query' tab is active, showing the following SQL query:

```
1 | SELECT UPPER(airline_name) AS airline_name_upper
2 | FROM airline
3 | ORDER BY airline_id;
```

The 'Data Output' tab shows the results of the query. The table has one column, 'airline\_name\_upper', and 12 rows. The first seven rows are visible:

|   | airline_name_upper |
|---|--------------------|
| 1 | AIR ASTANA         |
| 2 | SCAT AIRLINES      |
| 3 | TURKISH AIRLINES   |
| 4 | BRITISH AIRWAYS    |
| 5 | GLOBAL AIRWAYS     |
| 6 | KAZAIR             |
| 7 | AIREASY            |

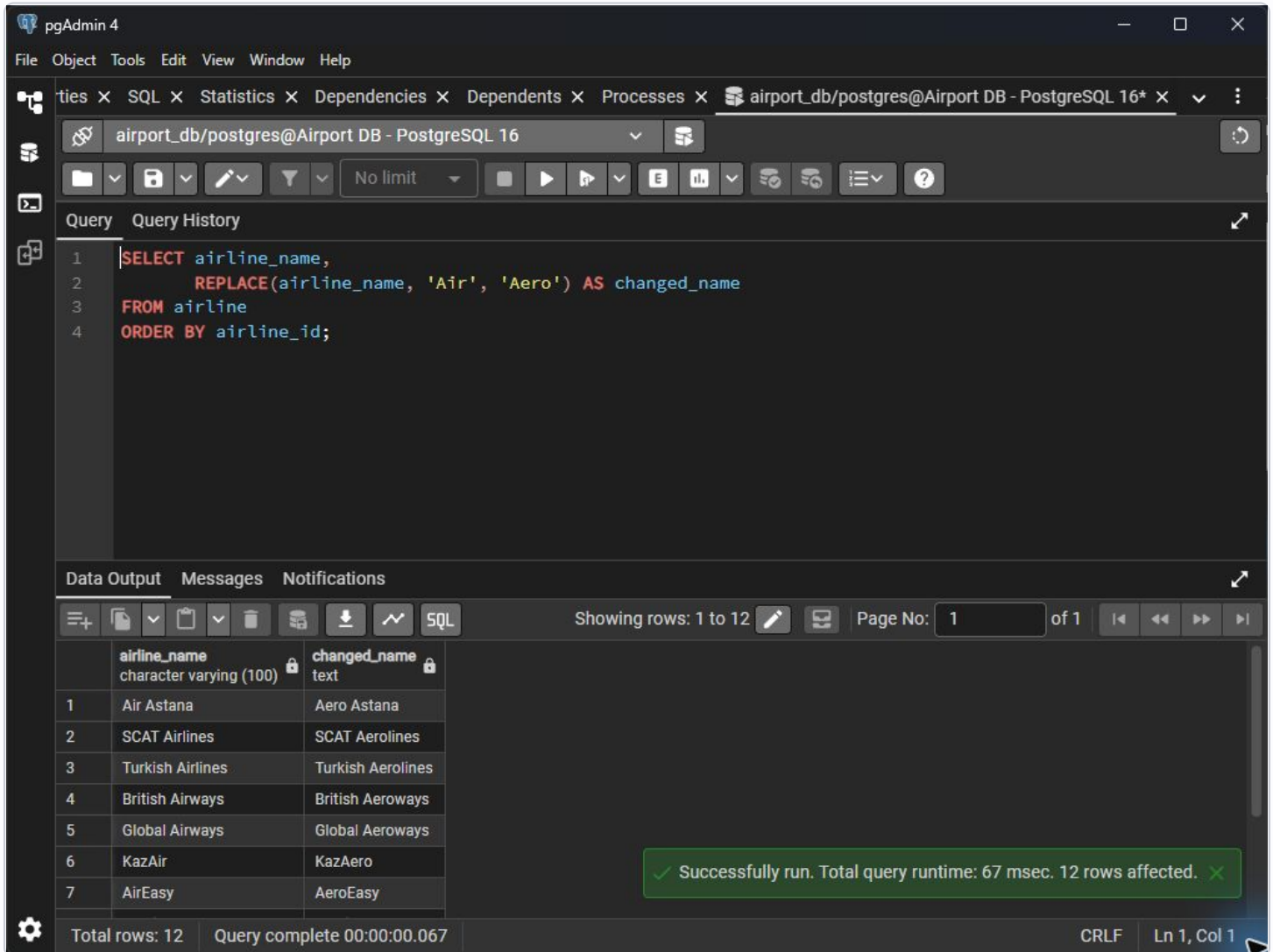
A green status bar at the bottom right indicates: 'Successfully run. Total query runtime: 57 msec. 12 rows affected.' The bottom status bar shows 'Total rows: 12', 'Query complete 00:00:00.057', and 'CRLF Ln 1, Col 1'.

## COMMAND EXPLANATION

**UPPER()** converts every letter in `airline_name` to uppercase. **AS** gives the result column a clear name. This is a **SELECT** query, so the original airline names in the table are not changed.

# Task 2

Replace the word Air in airline names with Aero.



The screenshot shows the pgAdmin 4 interface. The SQL editor contains the following query:

```
1 | SELECT airline_name,  
2 |        REPLACE(airline_name, 'Air', 'Aero') AS changed_name  
3 | FROM airline  
4 | ORDER BY airline_id;
```

The Data Output tab shows the results of the query. The table has two columns: `airline_name` (character varying (100)) and `changed_name` (text). The results are as follows:

|   | airline_name     | changed_name      |
|---|------------------|-------------------|
| 1 | Air Astana       | Aero Astana       |
| 2 | SCAT Airlines    | SCAT Aerolines    |
| 3 | Turkish Airlines | Turkish Aerolines |
| 4 | British Airways  | British Aeroways  |
| 5 | Global Airways   | Global Aeroways   |
| 6 | KazAir           | KazAero           |
| 7 | AirEasy          | AeroEasy          |

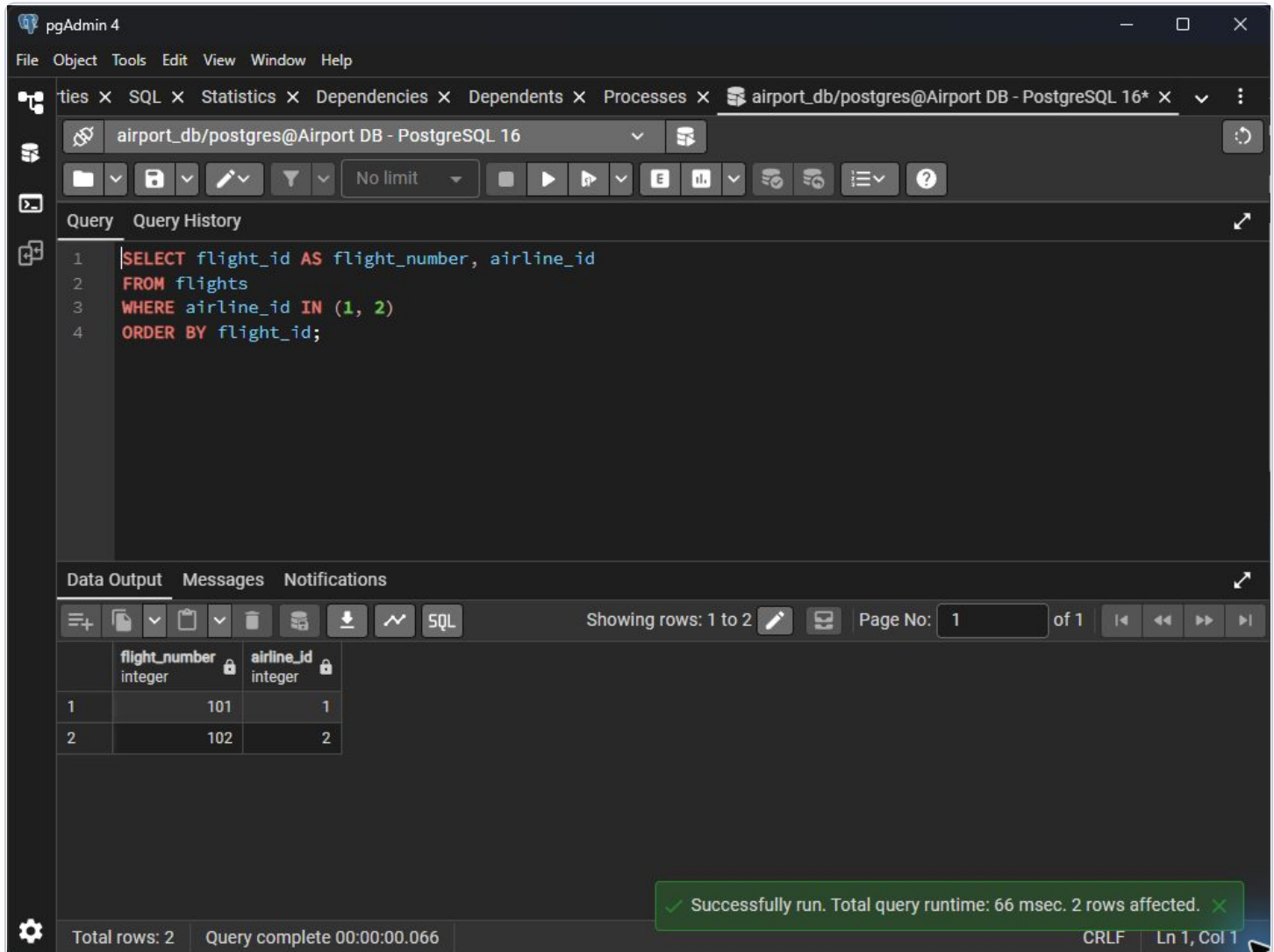
A green message box indicates: "Successfully run. Total query runtime: 67 msec. 12 rows affected." The status bar at the bottom shows "Total rows: 12" and "Query complete 00:00:00.067".

## COMMAND EXPLANATION

**REPLACE(text, old, new)** searches inside each name and substitutes the exact, case-sensitive text Air with Aero. The first column keeps the original value for comparison. No UPDATE is used, so stored data stays unchanged.

# Task 3

Find flight numbers connected with airline 1 or airline 2.



The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL query:

```
1 | SELECT flight_id AS flight_number, airline_id
2 | FROM flights
3 | WHERE airline_id IN (1, 2)
4 | ORDER BY flight_id;
```

The query results are displayed in the Data Output pane, showing 2 rows:

|   | flight_number<br>integer | airline_id<br>integer |
|---|--------------------------|-----------------------|
| 1 | 101                      | 1                     |
| 2 | 102                      | 2                     |

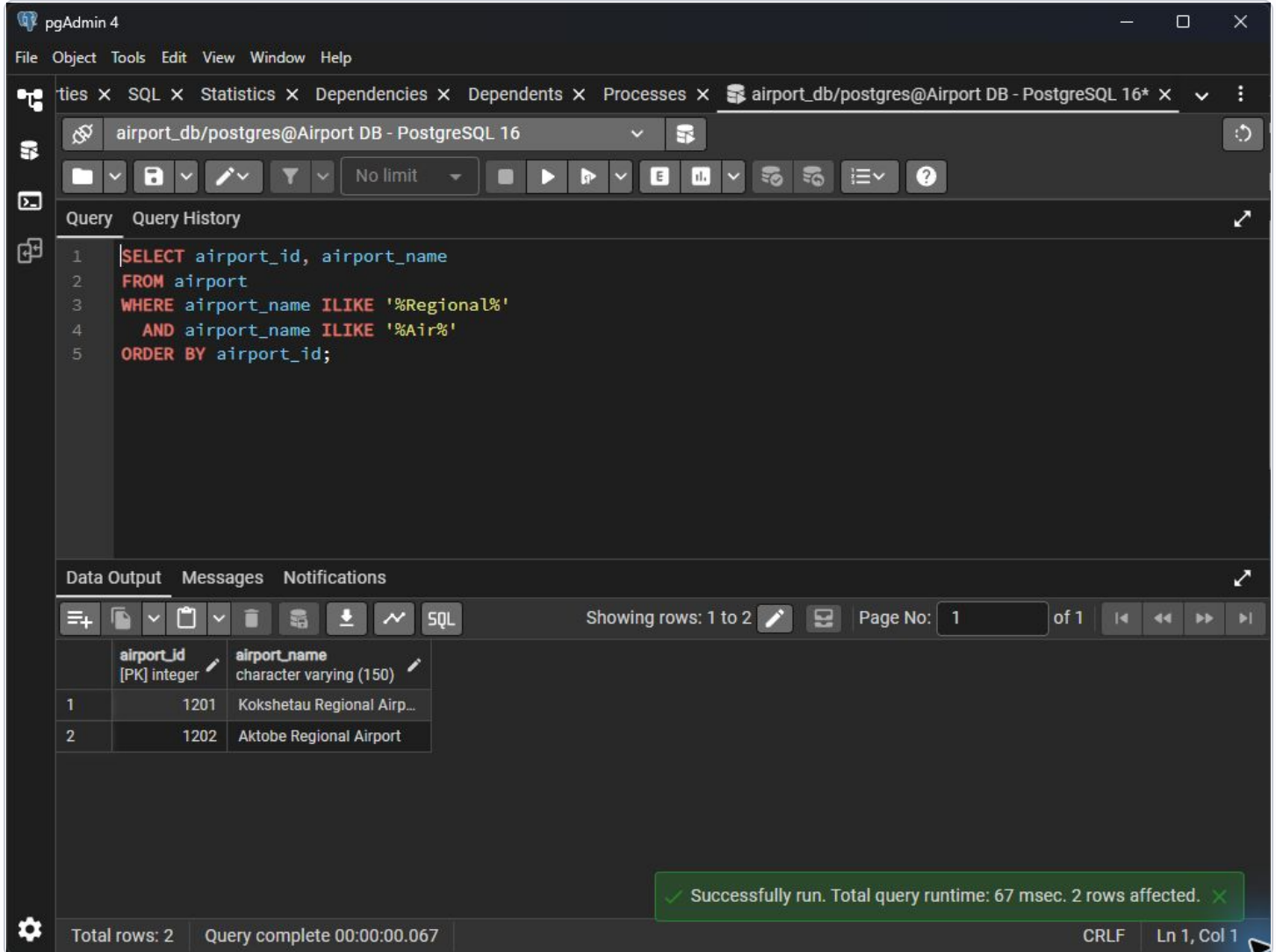
A status message at the bottom indicates: "Successfully run. Total query runtime: 66 msec. 2 rows affected."

## COMMAND EXPLANATION

IN (1, 2) is a short form of `airline_id = 1 OR airline_id = 2`. The current schema has no separate `flight_number` column, so `flight_id` is displayed under that name with AS.

# Task 4

Retrieve airports containing Regional and Air in the name.



The screenshot shows the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main toolbar contains various icons for database operations. The central pane displays a SQL query:

```
1 | SELECT airport_id, airport_name
2 | FROM airport
3 | WHERE airport_name ILIKE '%Regional%'
4 |   AND airport_name ILIKE '%Air%'
5 | ORDER BY airport_id;
```

Below the query editor, the 'Data Output' tab is active, showing the results of the query in a table format. The table has two columns: 'airport\_id' (integer, primary key) and 'airport\_name' (character varying (150)). The results are as follows:

| airport_id | airport_name               |
|------------|----------------------------|
| 1          | Kokshetau Regional Airp... |
| 2          | Aktobe Regional Airport    |

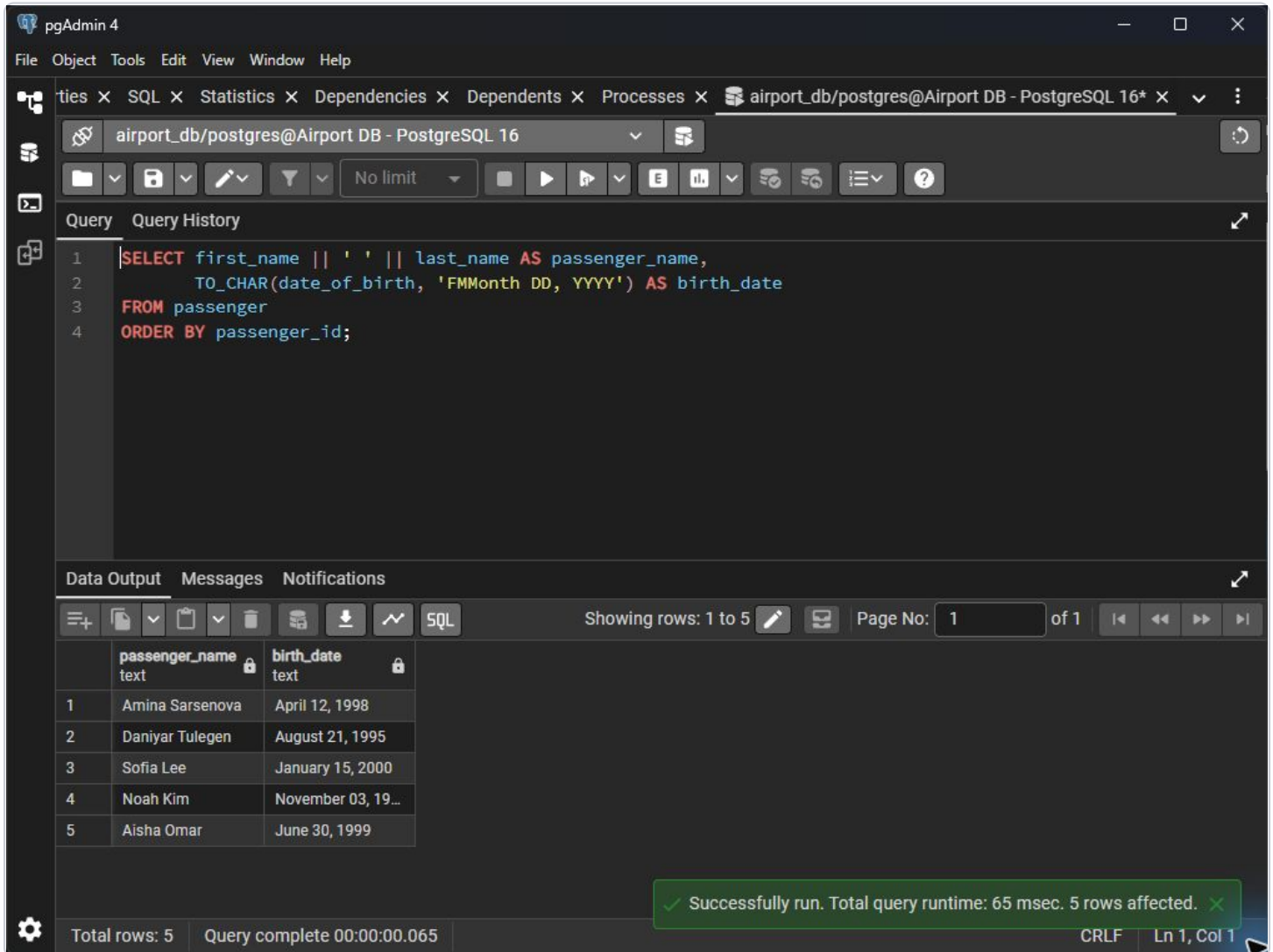
At the bottom of the interface, a status bar indicates 'Total rows: 2' and 'Query complete 00:00:00.067'. A green notification box at the bottom right states: 'Successfully run. Total query runtime: 67 msec. 2 rows affected.'

## COMMAND EXPLANATION

**ILIKE** performs a case-insensitive search. The percent signs mean that other characters may appear before or after the word. **AND** requires both conditions to be true. The assignment's spelling Regional was corrected to Regional.

# Task 5

Show passenger names and format their birth dates.



The screenshot shows the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main toolbar contains icons for connecting, refreshing, saving, and other database operations. The central pane displays a SQL query:

```
1 | SELECT first_name || ' ' || last_name AS passenger_name,  
2 |       TO_CHAR(date_of_birth, 'FMMonth DD, YYYY') AS birth_date  
3 | FROM passenger  
4 | ORDER BY passenger_id;
```

Below the query editor, the 'Data Output' tab is active, showing a table with 5 rows. The table has two columns: 'passenger\_name' (text) and 'birth\_date' (text). The data is as follows:

|   | passenger_name  | birth_date         |
|---|-----------------|--------------------|
| 1 | Amina Sarsenova | April 12, 1998     |
| 2 | Daniyar Tulegen | August 21, 1995    |
| 3 | Sofia Lee       | January 15, 2000   |
| 4 | Noah Kim        | November 03, 19... |
| 5 | Aisha Omar      | June 30, 1999      |

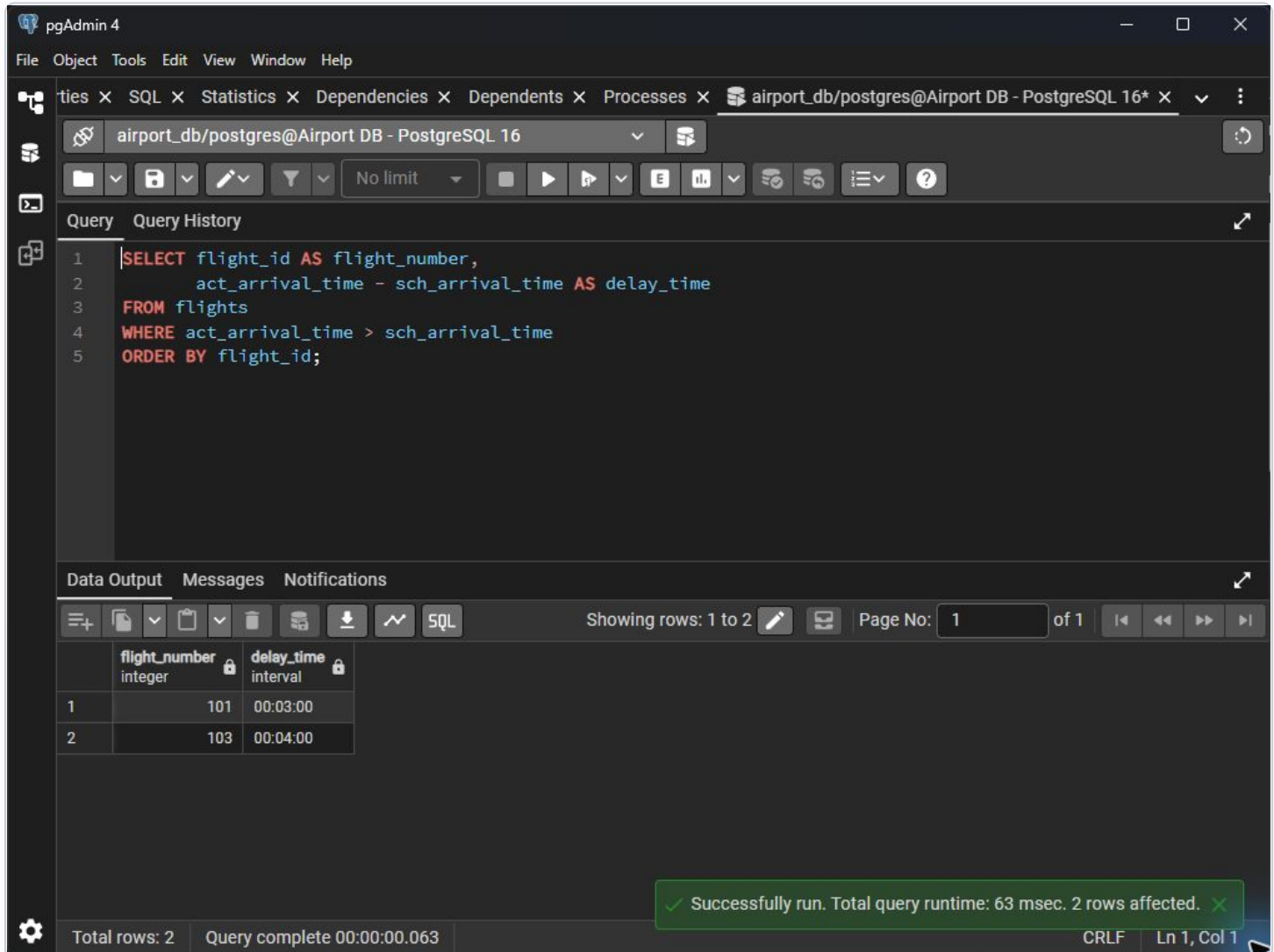
At the bottom, a green status bar indicates: 'Successfully run. Total query runtime: 65 msec. 5 rows affected.' The bottom left shows 'Total rows: 5' and 'Query complete 00:00:00.065'. The bottom right shows 'CRLF' and 'Ln 1, Col 1'.

## COMMAND EXPLANATION

The `||` operator joins `first_name`, a space, and `last_name`. `TO_CHAR` converts the date to text. `Month` prints the month name, `DD` the day, and `YYYY` the four-digit year; `FM` removes extra padding after the month.

# Task 6

Find delayed flight numbers and calculate the delay.



The screenshot shows the pgAdmin 4 interface. The SQL editor contains the following query:

```
1 | SELECT flight_id AS flight_number,  
2 |       act_arrival_time - sch_arrival_time AS delay_time  
3 | FROM flights  
4 | WHERE act_arrival_time > sch_arrival_time  
5 | ORDER BY flight_id;
```

The Data Output tab shows the results of the query:

|   | flight_number<br>integer | delay_time<br>interval |
|---|--------------------------|------------------------|
| 1 | 101                      | 00:03:00               |
| 2 | 103                      | 00:04:00               |

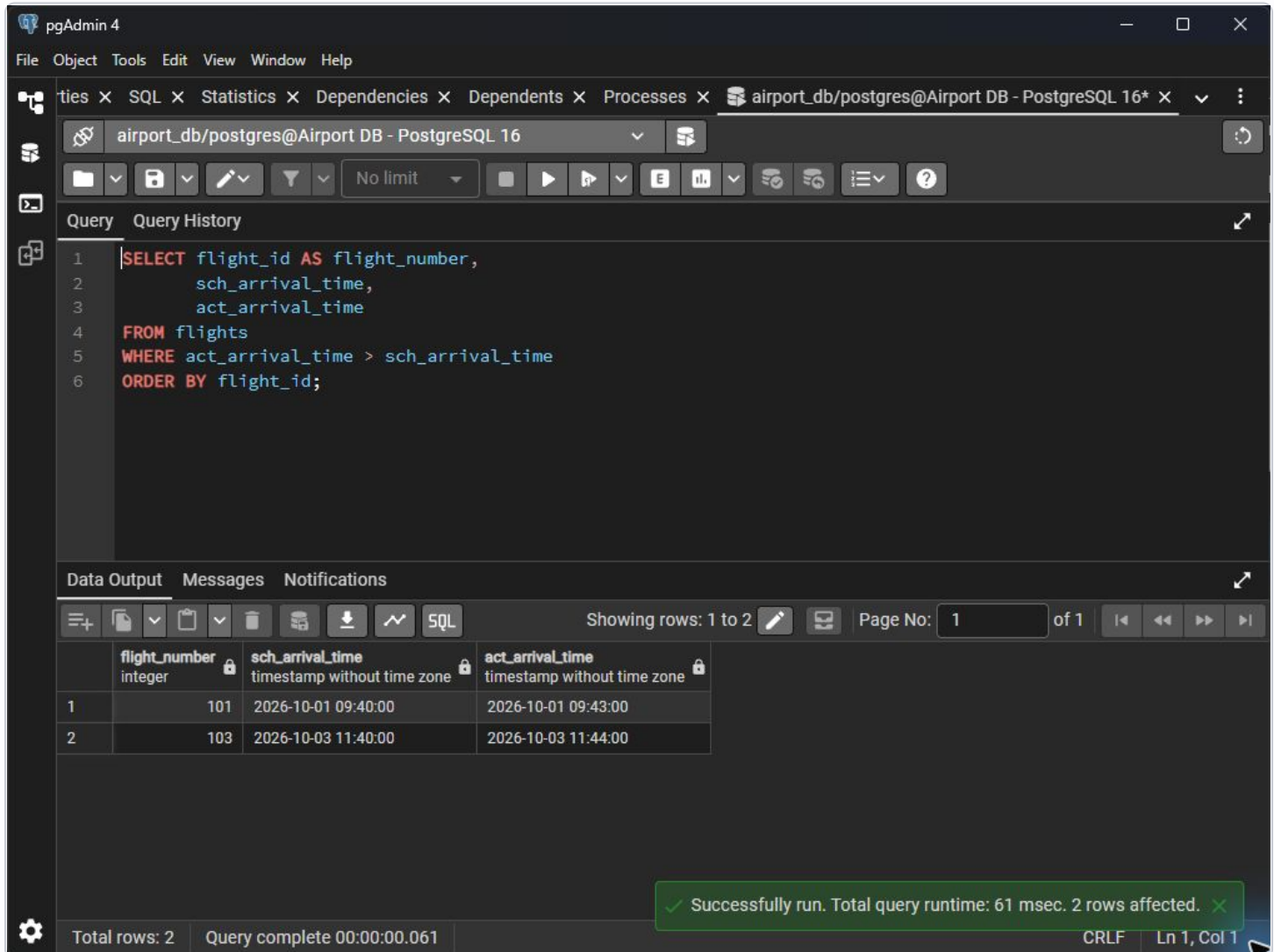
A green status bar at the bottom indicates: "Successfully run. Total query runtime: 63 msec. 2 rows affected."

## COMMAND EXPLANATION

A flight is delayed when its actual arrival is later than its scheduled arrival. Subtracting the two PostgreSQL timestamps returns an interval, which shows the exact delay: three minutes for flight 101 and four minutes for flight 103.

# Task 7

List flights that arrived later than scheduled.



The screenshot shows the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window displays the 'airport\_db/postgres@Airport DB - PostgreSQL 16' connection. The 'Query' tab is active, showing the following SQL query:

```
1 | SELECT flight_id AS flight_number,  
2 |       sch_arrival_time,  
3 |       act_arrival_time  
4 | FROM flights  
5 | WHERE act_arrival_time > sch_arrival_time  
6 | ORDER BY flight_id;
```

The 'Data Output' tab shows the results of the query. The table has three columns: flight\_number (integer), sch\_arrival\_time (timestamp without time zone), and act\_arrival\_time (timestamp without time zone). The results are as follows:

|   | flight_number<br>integer | sch_arrival_time<br>timestamp without time zone | act_arrival_time<br>timestamp without time zone |
|---|--------------------------|-------------------------------------------------|-------------------------------------------------|
| 1 | 101                      | 2026-10-01 09:40:00                             | 2026-10-01 09:43:00                             |
| 2 | 103                      | 2026-10-03 11:40:00                             | 2026-10-03 11:44:00                             |

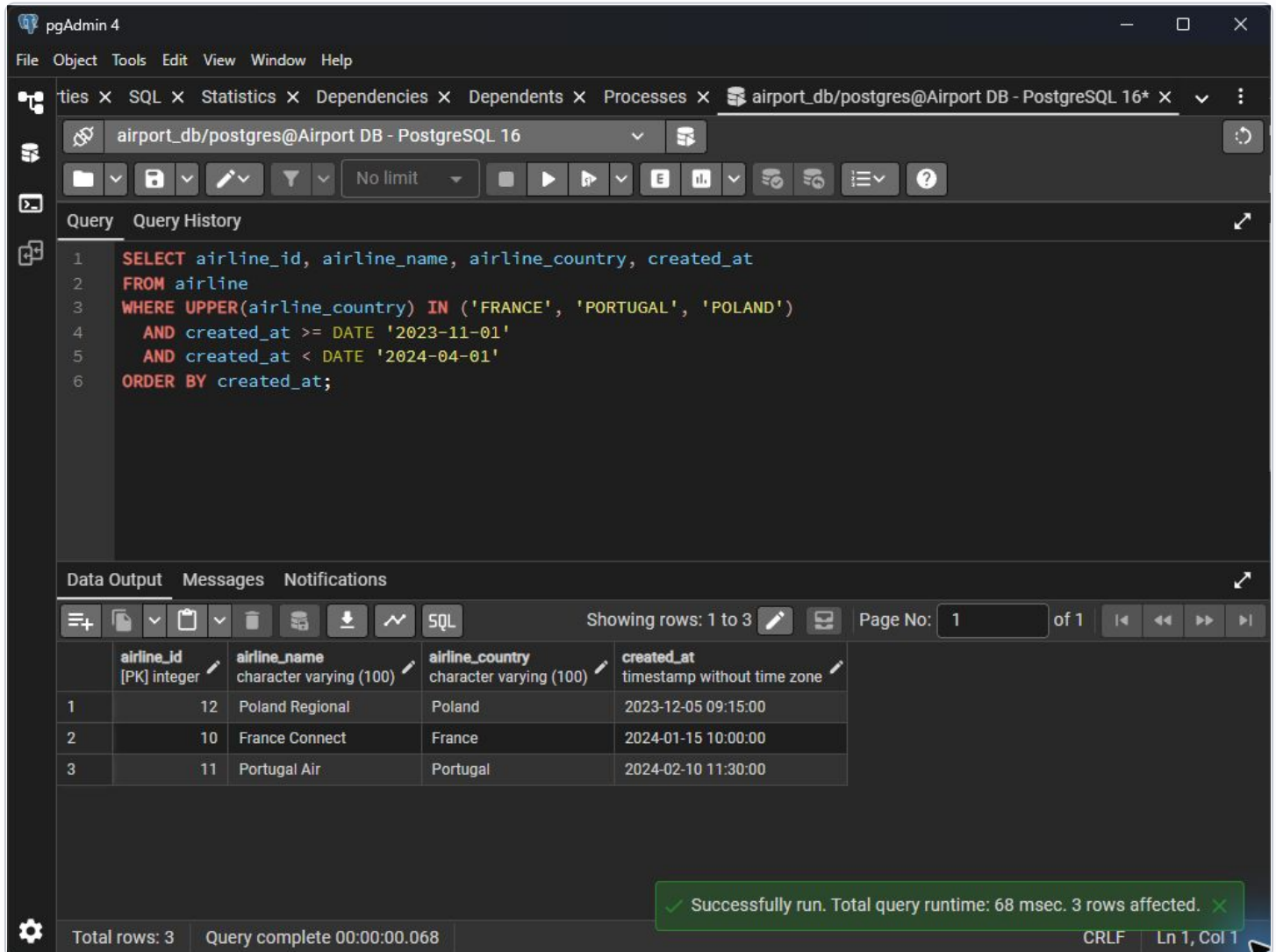
A green status bar at the bottom indicates: 'Successfully run. Total query runtime: 61 msec. 2 rows affected.' The bottom status bar shows 'Total rows: 2', 'Query complete 00:00:00.061', and 'CRLF Ln 1, Col 1'.

## COMMAND EXPLANATION

The WHERE comparison keeps only rows where act\_arrival\_time is greater than sch\_arrival\_time. Both timestamps are displayed so the late arrival can be checked directly.

# Task 8

Show selected European airlines created in the given period.



The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'airport\_db/postgres@Airport DB - PostgreSQL 16\*' database. The query selects airlines from the 'airline' table based on country and creation date.

```
1 SELECT airline_id, airline_name, airline_country, created_at
2 FROM airline
3 WHERE UPPER(airline_country) IN ('FRANCE', 'PORTUGAL', 'POLAND')
4       AND created_at >= DATE '2023-11-01'
5       AND created_at < DATE '2024-04-01'
6 ORDER BY created_at;
```

The results are displayed in a table with 5 columns: **airline\_id** (integer), **airline\_name** (character varying), **airline\_country** (character varying), and **created\_at** (timestamp without time zone). The table shows 3 rows of data.

|   | airline_id | airline_name    | airline_country | created_at          |
|---|------------|-----------------|-----------------|---------------------|
| 1 | 12         | Poland Regional | Poland          | 2023-12-05 09:15:00 |
| 2 | 10         | France Connect  | France          | 2024-01-15 10:00:00 |
| 3 | 11         | Portugal Air    | Portugal        | 2024-02-10 11:30:00 |

A green status bar at the bottom indicates: "Successfully run. Total query runtime: 68 msec. 3 rows affected."

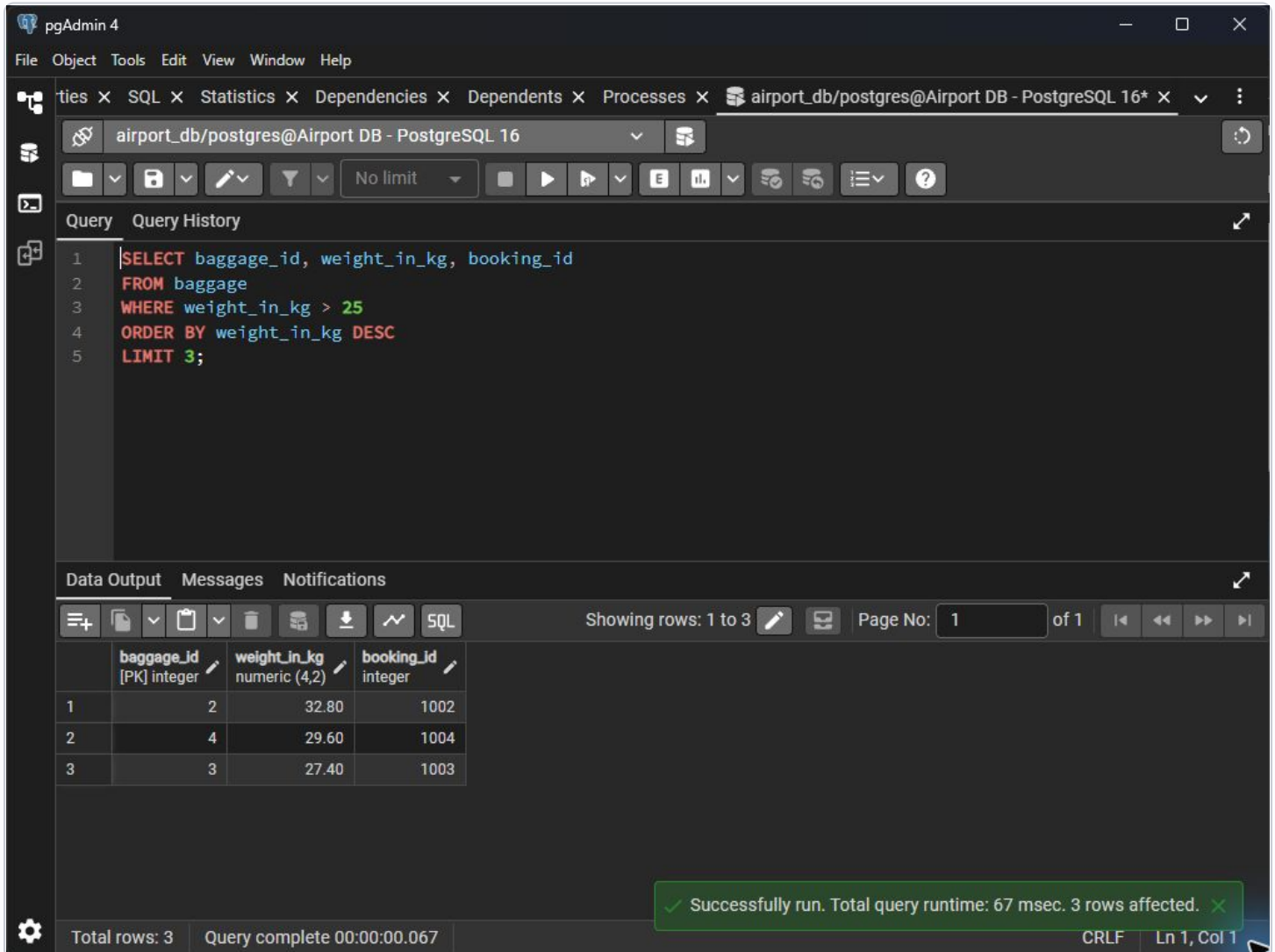
Summary: Total rows: 3, Query complete 00:00:00.068, CRLF, Ln 1, Col 1

## COMMAND EXPLANATION

**IN** selects any country from the list, while **UPPER** makes different letter cases harmless. The period uses a half-open range: it starts on November 1 and stops before April 1, so every time on March 31 is included.

# Task 9

Find the top three baggage items heavier than 25 kg.



The screenshot shows the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main toolbar contains various icons for database operations. The central pane displays a SQL query:

```
1 | SELECT baggage_id, weight_in_kg, booking_id
2 | FROM baggage
3 | WHERE weight_in_kg > 25
4 | ORDER BY weight_in_kg DESC
5 | LIMIT 3;
```

Below the query editor, the 'Data Output' tab is active, showing the results of the query in a table format. The table has three columns: baggage\_id [PK] integer, weight\_in\_kg numeric (4,2), and booking\_id integer. The results are as follows:

|   | baggage_id [PK] integer | weight_in_kg numeric (4,2) | booking_id integer |
|---|-------------------------|----------------------------|--------------------|
| 1 | 2                       | 32.80                      | 1002               |
| 2 | 4                       | 29.60                      | 1004               |
| 3 | 3                       | 27.40                      | 1003               |

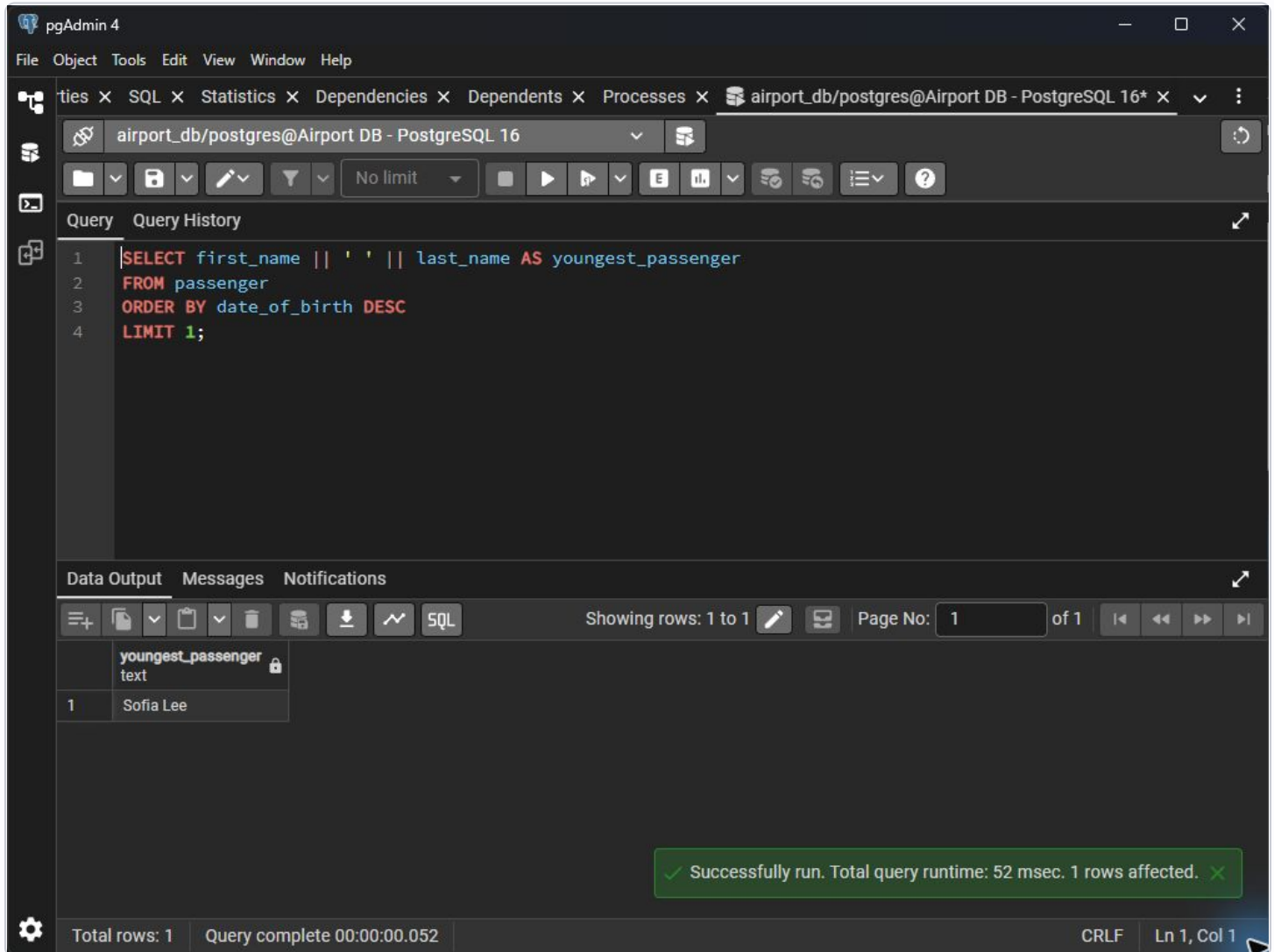
At the bottom of the interface, a status bar indicates 'Total rows: 3' and 'Query complete 00:00:00.067'. A green notification box at the bottom right states: 'Successfully run. Total query runtime: 67 msec. 3 rows affected.'

## COMMAND EXPLANATION

**WHERE** removes baggage weighing 25 kg or less. **ORDER BY ... DESC** places the heaviest item first. **LIMIT 3** leaves only the first three rows after sorting.

# Task 10

Find the youngest passenger's full name.



The screenshot shows the pgAdmin 4 interface. The SQL editor contains the following query:

```
1 | SELECT first_name || ' ' || last_name AS youngest_passenger
2 | FROM passenger
3 | ORDER BY date_of_birth DESC
4 | LIMIT 1;
```

The Data Output pane shows the result of the query:

|   | youngest_passenger<br>text |
|---|----------------------------|
| 1 | Sofia Lee                  |

A green status bar at the bottom indicates: "Successfully run. Total query runtime: 52 msec. 1 rows affected."

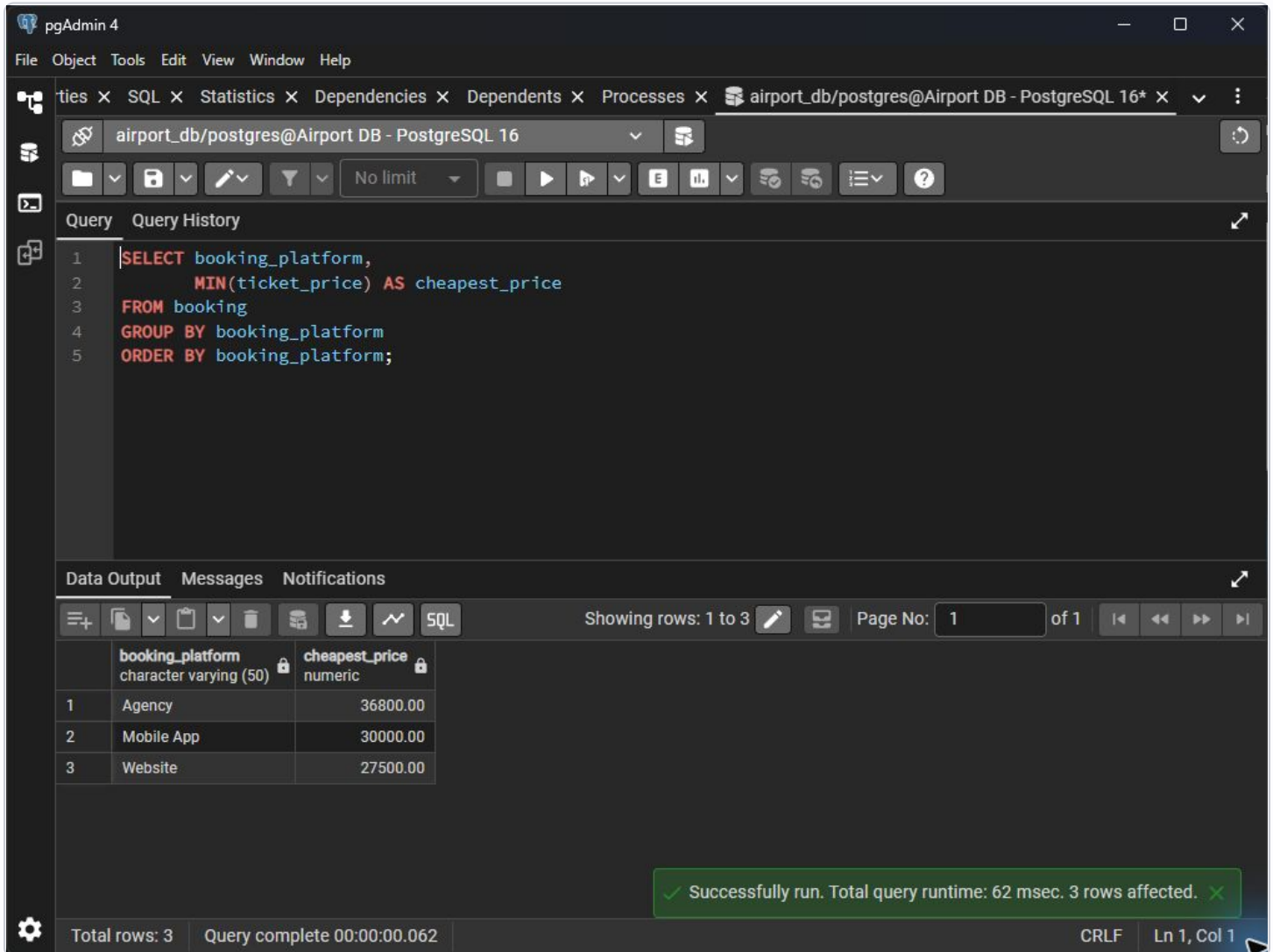
At the bottom of the interface, it shows "Total rows: 1" and "Query complete 00:00:00.052".

## COMMAND EXPLANATION

For dates of birth, the latest date belongs to the youngest person. DESC puts that date first, and LIMIT 1 returns one passenger. The two name columns are joined with a space.

# Task 11

Find the cheapest booking price on each platform.



The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'airport\_db/postgres@Airport DB - PostgreSQL 16\*' database. The query is:

```
1 | SELECT booking_platform,  
2 |        MIN(ticket_price) AS cheapest_price  
3 | FROM booking  
4 | GROUP BY booking_platform  
5 | ORDER BY booking_platform;
```

The results are displayed in the 'Data Output' tab, showing 3 rows:

|   | booking_platform<br>character varying (50) | cheapest_price<br>numeric |
|---|--------------------------------------------|---------------------------|
| 1 | Agency                                     | 36800.00                  |
| 2 | Mobile App                                 | 30000.00                  |
| 3 | Website                                    | 27500.00                  |

A green status bar at the bottom indicates: 'Successfully run. Total query runtime: 62 msec. 3 rows affected.'

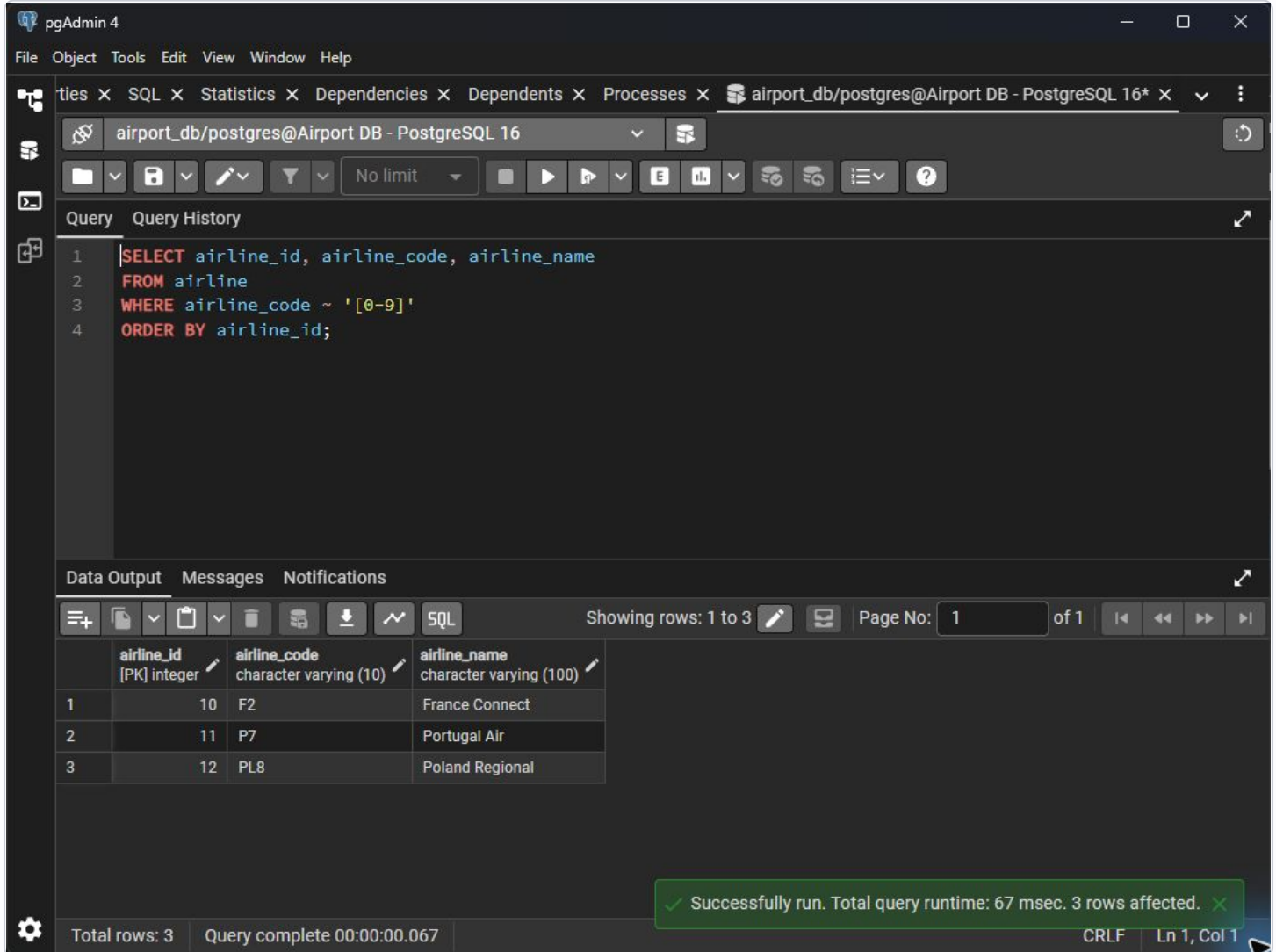
At the bottom left, it shows 'Total rows: 3' and 'Query complete 00:00:00.062'. At the bottom right, it shows 'CRLF' and 'Ln 1, Col 1'.

## COMMAND EXPLANATION

**GROUP BY** creates one group for each booking platform. **MIN** then finds the smallest `ticket_price` inside every group, producing one result row per platform.

# Task 12

Return airlines whose airline code contains a digit.



The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL query:

```
1 | SELECT airline_id, airline_code, airline_name
2 | FROM airline
3 | WHERE airline_code ~ '[0-9]'
4 | ORDER BY airline_id;
```

The Data Output tab shows the results of the query, displaying 3 rows. The columns are `airline_id` (integer), `airline_code` (character varying (10)), and `airline_name` (character varying (100)).

|   | airline_id<br>[PK] integer | airline_code<br>character varying (10) | airline_name<br>character varying (100) |
|---|----------------------------|----------------------------------------|-----------------------------------------|
| 1 | 10                         | F2                                     | France Connect                          |
| 2 | 11                         | P7                                     | Portugal Air                            |
| 3 | 12                         | PL8                                    | Poland Regional                         |

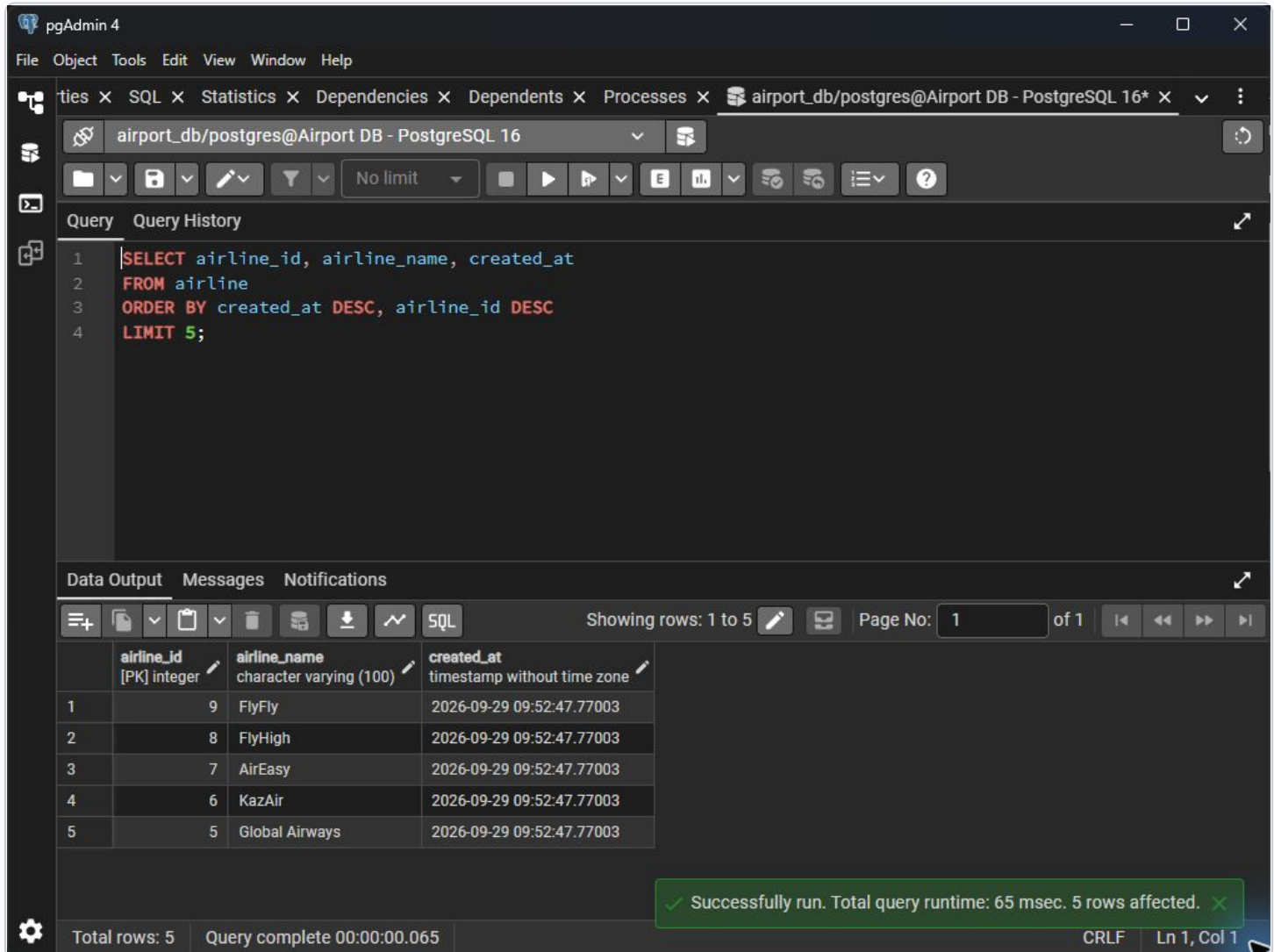
A green status bar at the bottom indicates: "Successfully run. Total query runtime: 67 msec. 3 rows affected." The bottom status bar shows "Total rows: 3", "Query complete 00:00:00.067", and "Ln 1, Col 1".

## COMMAND EXPLANATION

The PostgreSQL operator `~` checks a regular expression. The pattern `[0-9]` means any one digit from 0 through 9, so a code matches even when the digit appears in the middle or at the end.

# Task 13

List the five most recently created airlines.



The screenshot shows the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window has a tab for 'airport\_db/postgres@Airport DB - PostgreSQL 16\*'. Below the tab is a toolbar with various icons. The 'Query' tab is active, showing a SQL query:

```
1 | SELECT airline_id, airline_name, created_at
2 | FROM airline
3 | ORDER BY created_at DESC, airline_id DESC
4 | LIMIT 5;
```

The 'Data Output' tab is also active, displaying the results of the query in a table. The table has three columns: 'airline\_id' (integer), 'airline\_name' (character varying (100)), and 'created\_at' (timestamp without time zone). The results show five rows, all with the same 'created\_at' timestamp of '2026-09-29 09:52:47.77003'.

|   | airline_id<br>[PK] integer | airline_name<br>character varying (100) | created_at<br>timestamp without time zone |
|---|----------------------------|-----------------------------------------|-------------------------------------------|
| 1 | 9                          | FlyFly                                  | 2026-09-29 09:52:47.77003                 |
| 2 | 8                          | FlyHigh                                 | 2026-09-29 09:52:47.77003                 |
| 3 | 7                          | AirEasy                                 | 2026-09-29 09:52:47.77003                 |
| 4 | 6                          | KazAir                                  | 2026-09-29 09:52:47.77003                 |
| 5 | 5                          | Global Airways                          | 2026-09-29 09:52:47.77003                 |

At the bottom of the interface, a status bar shows 'Total rows: 5' and 'Query complete 00:00:00.065'. A green notification box at the bottom right states: 'Successfully run. Total query runtime: 65 msec. 5 rows affected.'

## COMMAND EXPLANATION

Sorting `created_at` in descending order puts the newest timestamps first. `airline_id DESC` is a tie-breaker when timestamps are equal. `LIMIT 5` returns only the first five rows.

# Task 14

Find unchecked baggage rows for booking IDs from 200 to 300.

The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL query:

```
1 SELECT baggage_check_id AS check_id,  
2       check_result,  
3       created_at,  
4       updated_at,  
5       booking_id,  
6       passenger_id  
7 FROM baggage_check  
8 WHERE booking_id BETWEEN 200 AND 300  
9       AND check_result <> 'Checked'  
10 ORDER BY baggage_check_id;
```

The Data Output tab shows the results of the query. The table has 6 columns: check\_id (integer), check\_result (character varying (30)), created\_at (timestamp without time zone), updated\_at (timestamp without time zone), booking\_id (integer), and passenger\_id (integer). The results are as follows:

|   | check_id<br>integer | check_result<br>character varying (30) | created_at<br>timestamp without time zone | updated_at<br>timestamp without time zone | booking_id<br>integer | passenger_id<br>integer |
|---|---------------------|----------------------------------------|-------------------------------------------|-------------------------------------------|-----------------------|-------------------------|
| 1 | 200                 | Not checked                            | 2024-03-20 12:00:00                       | 2024-03-18 09:00:00                       | 220                   | 1                       |
| 2 | 201                 | Inspection                             | 2024-04-02 11:00:00                       | 2024-04-02 11:15:00                       | 250                   | 2                       |

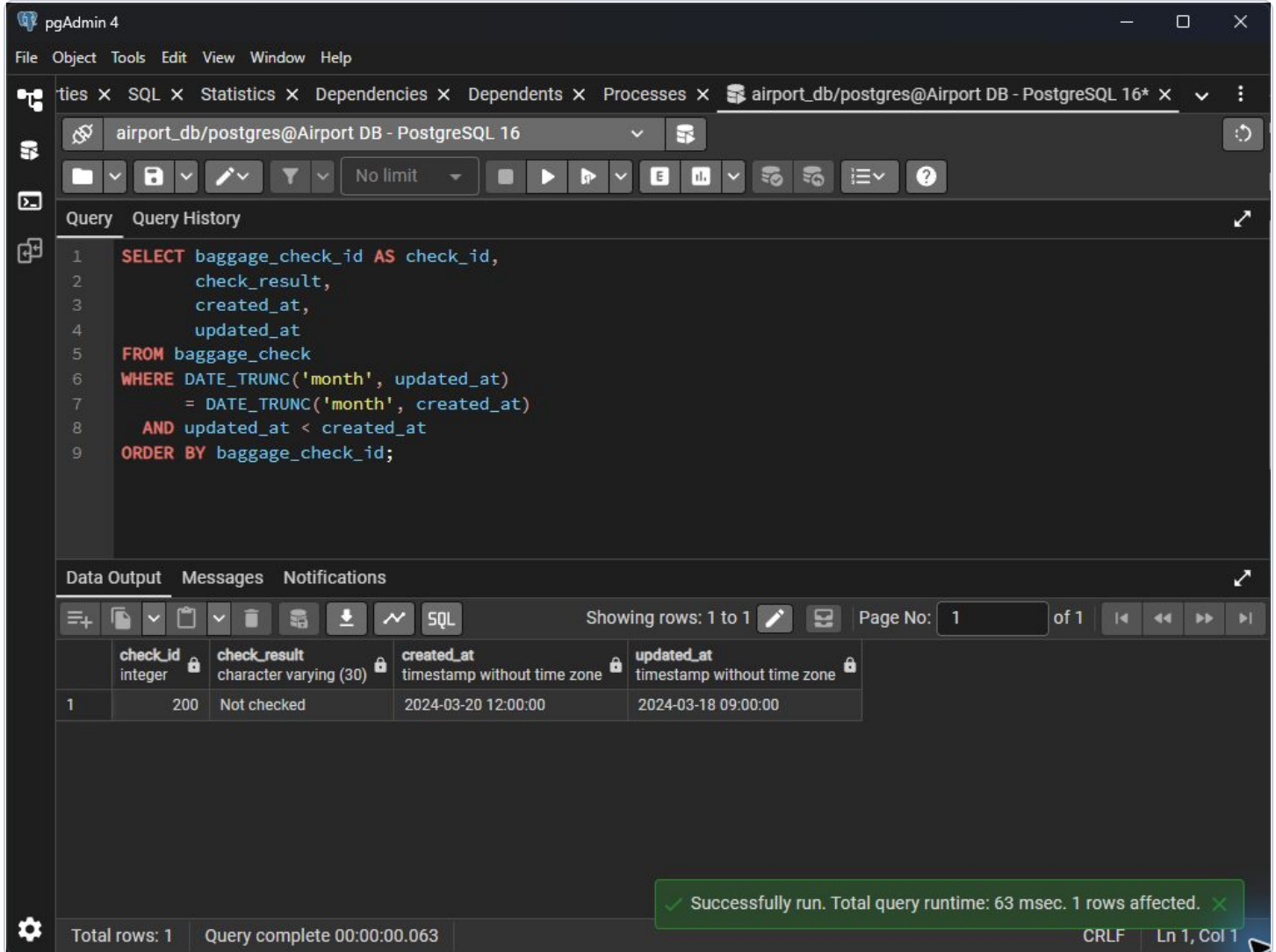
A green status bar at the bottom indicates: "Successfully run. Total query runtime: 64 msec. 2 rows affected." The bottom status bar shows "Total rows: 2", "Query complete 00:00:00.064", and "Ln 1, Col 1".

## COMMAND EXPLANATION

**BETWEEN** includes both boundary values, so 200 and 300 are part of the range. The **<>** operator means not equal; therefore rows whose result is exactly Checked are excluded.

# Task 15

Find baggage checks updated earlier in their creation month.



The screenshot shows the pgAdmin 4 interface. The SQL query editor contains the following query:

```
1 SELECT baggage_check_id AS check_id,  
2       check_result,  
3       created_at,  
4       updated_at  
5 FROM baggage_check  
6 WHERE DATE_TRUNC('month', updated_at)  
7       = DATE_TRUNC('month', created_at)  
8       AND updated_at < created_at  
9 ORDER BY baggage_check_id;
```

The Data Output tab shows the results of the query. The table has 4 columns: check\_id (integer), check\_result (character varying (30)), created\_at (timestamp without time zone), and updated\_at (timestamp without time zone). The results show 1 row:

| check_id | check_result | created_at          | updated_at          |
|----------|--------------|---------------------|---------------------|
| 200      | Not checked  | 2024-03-20 12:00:00 | 2024-03-18 09:00:00 |

A green status bar at the bottom indicates: "Successfully run. Total query runtime: 63 msec. 1 rows affected."

## COMMAND EXPLANATION

`DATE_TRUNC('month', ...)` removes the day and time, allowing the two months to be compared. The second condition finds the data error where `updated_at` is earlier than `created_at`. The real column name is `updated_at`, not `update_at`.